

Self-Training with Structured Data for Efficient Insect Pest Detection

Bachelor's Thesis

Christoph Wald

Matrikelnummer 17515905

christoph.wald@stud.uni-goettingen.de

Supervisors:

Prof. Alexander Ecker (Georg-August-Universität Göttingen)

Dr. Elias Böckmann (Julius-Kühn-Institut Braunschweig)

October 29, 2025

Contents

1	Introduction	2
2	Methods	6
2.1	Overview	6
2.2	Data	6
2.2.1	Dataset description	6
2.2.2	Preprocessing	10
2.2.3	Splitting	12
2.3	Evaluation	12
2.4	Supervised training	14
2.5	Image processing	16
2.5.1	Preprocessing for segmentation	16
2.5.2	Segmentation	19
2.5.3	Augmenting the data	22
2.6	Self-Training	23
2.6.1	Training on the labels created by image processing	23
2.6.2	Further training on predicted labels	24
3	Results	24
3.1	Comparing supervised training and self-training	24
3.2	Evaluation settings	25
3.3	Supervised training	25
3.4	Image processing	28
3.4.1	Preprocessing for segmentation	28

3.4.2 Segmentation	31
3.5 Training on the labels created by image processing	32
4 Discussion	35
5 Conclusion	35
6 References	35
7 Glossary	38

1 Introduction

Insect pests are responsible for significant production losses in agriculture [1]. However, pesticides threaten the environment by reducing biodiversity [2], particularly among insects, including pollinators that support agricultural production [3]. Overuse of pesticides causes resistance in pests. Additionally, global warming leads to increasing pest populations, which aggravates the problem further [1]. To address these challenges, the concept of integrated pest management (IPM) was developed. As defined by the Food and Agriculture Organization of the United Nations (FAO), it sets the goal of minimizing the use of pesticides by applying alternative management strategies wherever possible [4,5]. IPM is also the legal basis for the use of pesticides in the EU [6], which, among other things, prescribes the monitoring of insect pests when using pesticides. Precise monitoring facilitates the early and targeted use of pesticides for only small infested patches, the use of less harmful pesticides, or alternative ways of control like beneficial insects. Monitoring is usually based on setting up a grid of sticky traps, which are regularly checked for trapped insects. Because the manual counting requires expertise and takes a lot of time, it is not feasible for larger applications [7]. Hence, computer vision-based systems for automated monitoring of insect pests are considered to be part of the solution. Ongoing efforts to automatize insect pest monitoring have more recently adopted the methods of deep learning (see the surveys Teixeira et al. 2023 [8] and Mittal et al. 2024 [9]). My project aims to facilitate the implementation of deep learning-based, automated insect pest monitoring.

The dominant approach in pest monitoring in general involves the supervised training of YOLO models. YOLO ("You Only Look Once") is a deep learning architecture that unifies the tasks of object localization and classification into a single step. It was introduced

by Redmon et al. in 2016 [10] and has been improved on since then in several iterations by different contributors. For greenhouses, specifically designed neural network models have been employed successfully in 2021 [11,12], but the recent studies from 2023 onwards use YOLO models on individual datasets [13,14,15,16]. These six papers all report high performance, with mean average precision (mAP) or F1 scores ranging from 0.89 to 0.94.

Main challenges in the implementation are the small size of pests—which, even with high-resolution cameras, occupy only a very tiny fraction of an image—and the high degree of morphological variation among individuals. In less controlled environments additional complications arise from high morphological similarity between different species. These challenges create a demand for large annotated datasets; however, the collection and labeling of biological data require considerable effort and expertise. As a result, datasets are often small and exhibit other limitations, such as class imbalance. For an overview of these specific challenges, see Teixeira et al. (2023) [8]. As a remedy, the authors recommend semi-supervised learning (SSL), a family of methods for training with only partially data. Despite this, no survey reports applications of SSL in the field of pest monitoring in general and only one study in the context of the greenhouse makes use of SSL methods: Rustia et al. [17] proposed a self-training pipeline as a supplement to a fully supervised model as early as 2021. While the results were promising, the implementation was highly intricate, involving multi-stage setups with hand-tailored solutions. This complexity may explain why the approach has not been further explored. Outside of the greenhouse, in open-field settings, some efforts were made from 2024 onwards, either focusing on a single insect species with more diverse backgrounds or on a set of larger insects [18,19,20,21,22]. Another notable paper in this context is Dang et al. (2024) [23], who employ a field dataset of ten larger insect species and perform segmentation without manual annotation by leveraging a grounding language–image model. While extending this approach with classification and adapting it to the smaller insect pests typically found in greenhouses might be feasible, grounding models will not be considered in this study due to their limited speed and efficiency compared to YOLO-based methods.

Table 1 below shows an overview over the relevant literature. Green indicates a correspondence with the demands of my project, red the opposite, and orange overlappings. Because of the differing datasets and metrics, the results are not directly comparable and left out. Also not seen in the table are the pest classes included in the datasets: Only Rustia et. al [11] covered the same pest classes featured in this project. As of now no YOLO model has been trained to identify common greenhouse pests without relying on supervised training. Although working solutions for automatic insect pest monitoring in the greenhouse have been successfully tested, practical deployment stagnates and is hindered by the need for large, manually annotated datasets for new target domains. To my knowledge, there is no adequate method to reduce the amount of labels needed. The

goal of the thesis is therefore an improvement in implementation efficiency by adapting the labeling and training method.

Authors (Year)	Greenhouse	YOLO	Training
Li et al. 2021 [12]	yes	no	supervised
Rustia et al. 2021 [11]	yes	no	supervised
Rustia et al. 2021 [17]	yes	no	SSL (predicted pseudo-labels by separate pipeline)
Costa et al. 2021 [13]	yes	yes	supervised
Zhang et al. 2023 [14]	yes	yes	supervised
Li et al. 2024 [18]	no	no	SSL (teacher-student)
Majewski et al. 2024 [19]	no	yes	SSL (pseudo-labels)
Wang et al. 2024 [15]	yes	yes	supervised
Zhou et al. 2024 [20]	no	no	SSL(teacher-student)
Gomez-Zamanillo et al. 2025 [22]	no	no	supervised, labels enhanced by image processing
Shi et al. 2025 [16]	yes	yes	supervised
Zhou and Shi 2025 [21]	yes	no	SSL(teacher-student)

Table 1: Overview over relevant papers. Colors indicate overlaps to my study (green: identical, orange: overlap, and red: different; yes/no indicate if the respective paper is settled in the greenhouse context or uses a YOLO-model respectively). None of these studies uses a simple self-training approach and none tries any type of SSL with a YOLO-model in the greenhouse setting.

I propose that given a specifically structured training dataset, a model can be trained without the need for manual labeling. This is achieved by a combination of image processing and a semi-supervised learning method (SSL) called self-training. The results are expected to contribute to a software foundation for a low-cost, open-source insect pest monitoring solution. It should be trainable on a single GPU and also provide efficient inference. Such a system could support practical deployment in greenhouse production settings and serve as a research tool for further studies.

The key requirement for the proposed solution is the specific structure of the dataset, which should be partitioned into subsets. All images in one subset should only contain one specific pest class as objects. With already existing labels a supervised YOLO model is trained as reference point, which should achieve at least 0.9 precision and 0.8 recall per class. Precision is prioritized in this target values over recall for two reasons. First, recall can be improved later through denser placement of sticky traps, whereas precision depends primarily on model quality. Second, low precision risks erroneous pesticide use, which conflicts with the principles of IPM.

The envisioned adapted automated labeling and SSL is done in two stages. First, with image processing segmentation algorithms bounding boxes for the pest classes are created. Since the images are presorted to contain only one pest class, the detected bounding boxes can be directly associated with the corresponding label, yielding a partially labeled dataset. In the second stage, the partially labeled dataset as given by stage one is used in a self-training SSL framework with an adapted YOLO to produce a model.

ToDo: Update this. At the moment the self-training does not work and I only created labels by image processing followed by one training with the created labels. So no SSL/self training really. But still without manual labels. Also adapt use case: It could be useful for implementing new insects to existing models.

2 Methods

2.1 Overview

:

- code is here: https://github.com/ChristophWald/insect_pest_detection (documentation is lagging)
- Data was prepared for experiments by cleaning and splitting into set 2.2
- Evaluation pipeline was set up 2.3
- Supervised training on manual labels was done as a baseline 2.4
- labels and training data were generated by image processing 2.5
- a model was trained on the generated labels, the model was used to predict new labels for more training iterations 2.6

2.2 Data

2.2.1 Dataset description

The data is provided by the Julius-Kühn-Institut (JKI) and was collected as part of the Smart-Checkpots project [24]. It consists of four subsets of images each associated with one of four insect pest classes (see Table 2). Every image in a subset is supposed to depict several instances of only one pest class trapped on a yellow sticky trap (YST)¹ (see Figure 1).

¹In general, there are different types of traps e.g. pheromone traps and there are also sticky traps with different colours, but YSTs are the most commonly used traps in greenhouses. Yellow is a compromise which most attract most insect pests.

Common Name	Scientific Name	EPPO Code
Fungus gnats	<i>Bradysia impatiens</i> , JOHANNSEN 1912 (Diptera: Sciarioidea)	BRAIIM
Tomato leaf miner	<i>Liriomyza bryoniae</i> , KALTENBACH 1858 (Diptera: Agromyzidae)	LIRIBO
Western flower thrips	<i>Frankliniella occidentalis</i> , PERGANDE 1895 (Thysanoptera: Thripidae)	FRANOC
Greenhouse whitefly	<i>Trialeurodes vaporariorum</i> , WESTWOOD 1856 (Hemiptera: Aleyrodidae)	TRIAVA

Table 2: Overview of insect pest classes present in the dataset.

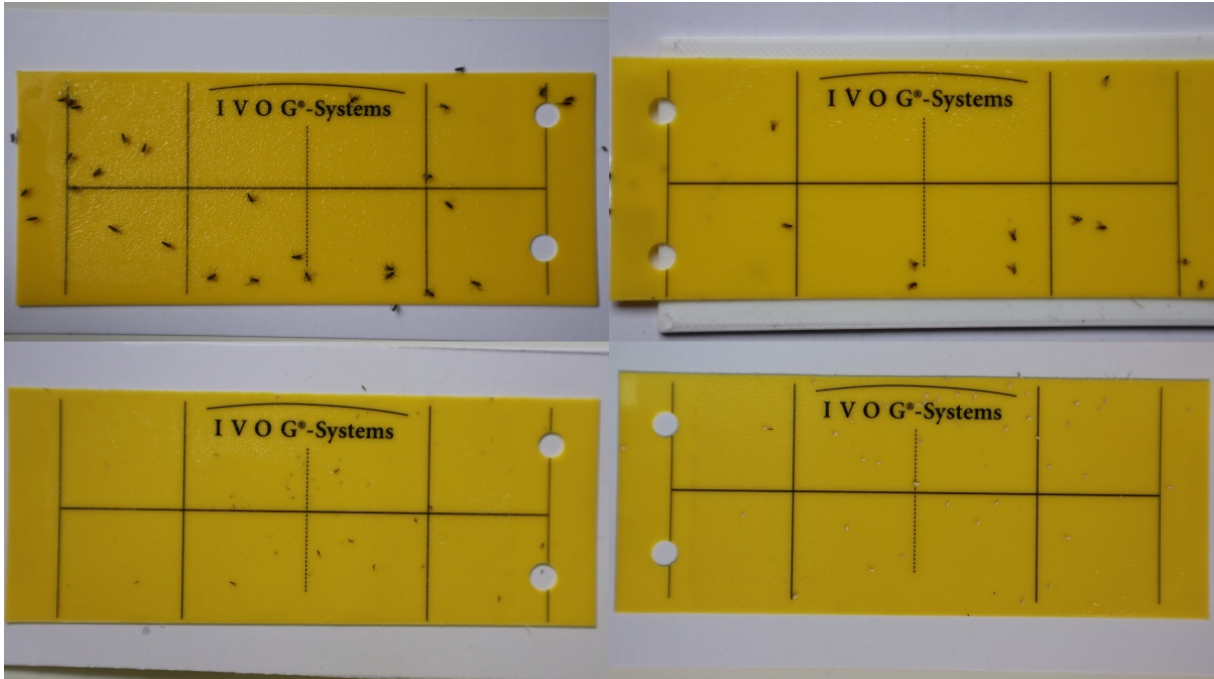


Figure 1: Example images from the four subsets with images from fungus gnats, leaf miner flies, thrips, and whiteflies (left to right, top to bottom). Reduced resolution.

The chosen insect pests are common in greenhouses, particularly on crops such as tomatoes and cucumbers, where they can cause severe damage, up to complete crop loss [11]. As Figure 2 shows, the size of the pest classes and thereby the number of pixels they cover differs considerable. In thrips, also male and female differ in appearance. The female individuals are larger and have good contrast against the yellow background, whereas the smaller and more transparent males are harder to detect (Row 3, box 3,7, and 9).

TODO: Change Row3/Image1

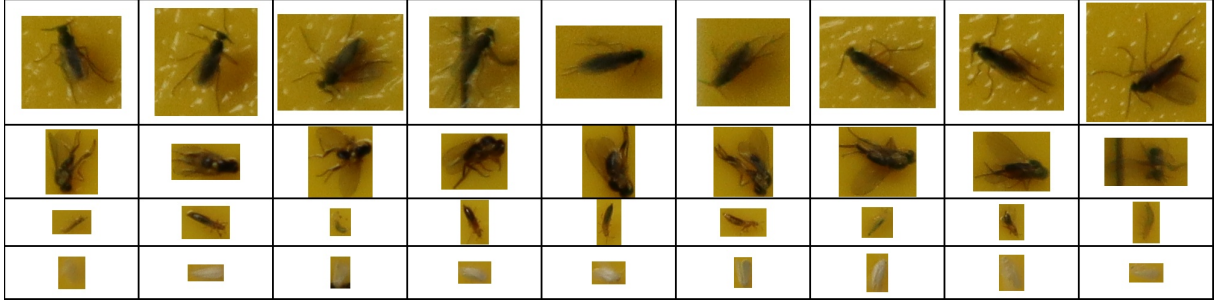


Figure 2: Cropped boxes with fungus gnats, leaf miner flies, thrips, and whiteflies (top to bottom). Original resolution.

The populated YSTs in this dataset were created at the JKI by breeding the insects to the adult stage in a closed environment (see Figure 3a) and exposing the YST in the environment for some time until it became populated. The traps were then immediately attached onto a piece of white paper, and pictures were taken with a Canon EOS M50 Mark II on a tripod at a distance of approximately 13 cm (see Figure 3b) and a resolution of 6000×3368 pixels, resulting in files of about 5–7 MB. No artificial light was used to avoid reflections. The pictures have slightly different view angles and different lighting conditions, some still affected from reflections and some having areas with low sharpness. Some of these aspects may be also be part of the real data to be processed after training.



(a) Breeding station



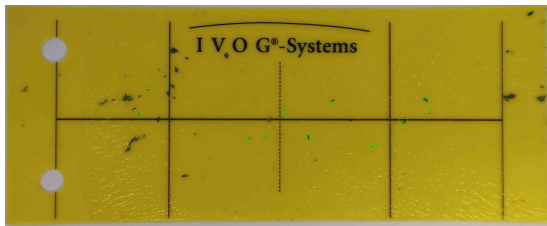
(b) Camera setup

Figure 3: Data creation, pictures with kind permission M. Pollmann, JKI

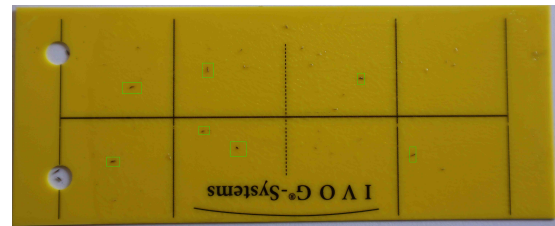
A more thorough explanation for the exclusion of the thrips is needed (and in the next section there is also paragraph on this). How was the catching done, why couldn't it be done otherwise? Is it possible to distinguish between larvae and male thrips or is it more of a guess? Remark that thrips larvae do not fly and will not be on the images in the final

use case. Also remark, that the final image has lower resolution, which will even more problematic. Give some more images of problematic thrips. Replace Figure 4b, as there are wrong labels on it. Maybe show images, were even Marcella was unsure.

On the images of the thrips subset sometimes larger amounts of foliage (see Figure 4a) are present. Also there are thrips in larval stages, which are hard to distinguish from the male individuals. This posed difficulties for the manual labeling, but is also a critical problem for the automated labeling process.



(a) Labeled thrips with foliage on the image



(b) Labeled thrips with large bounding boxes

Figure 4: Examples for thrips images and labels

For each pest class a proportion of images were manually labeled. Other than for the thrips, visual inspection showed the label quality to be consistent, although with a noticeable proportion unlabeled individuals for the fungus gnats. Also the size of the bounding boxes may differ for the batches that different persons labeled (see Figure 4), which will be taken into account.

Addition: Later on, the missing labels for the fungus gnats led to a high number of false positives and precision was comparably low although the model performed ok. Therefore the test set is checked for missing labels by the JKI right now, independently of results of course. See Figures below.

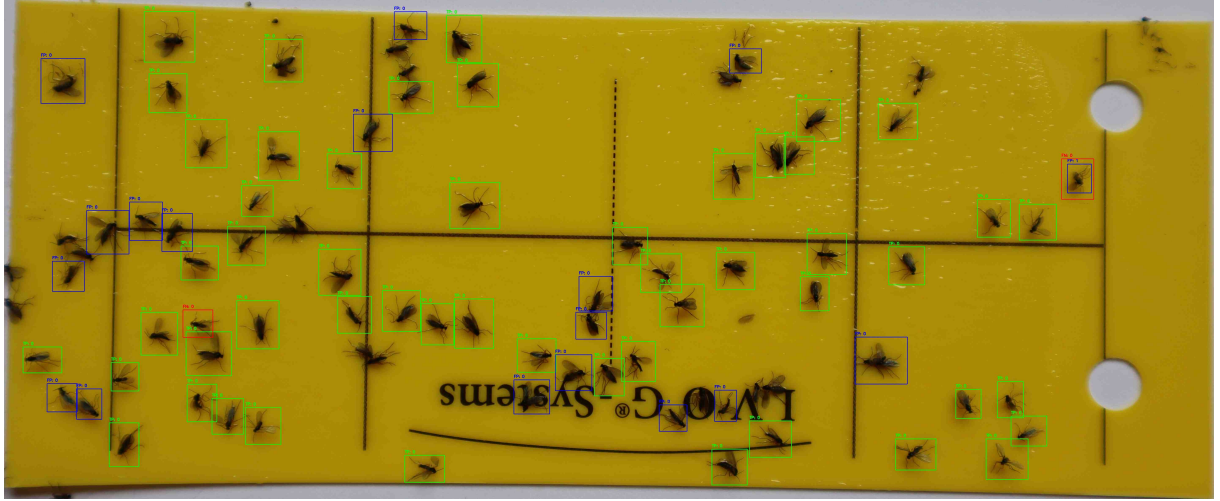


Figure 5: False positive fungus gnats (blue boxes) are actually correct and only considered false due to missing labels. Green boxes are true positives, red false negatives.



Figure 6: False positive labels with maximum confidences for fungus gnats (BRAIM), leaf miner flies (LIRIBO) and whiteflies (TRIAVA) (from top to bottom) In the fungus gnats all of this are actually correct but the labels are missing in the ground truth annotation.

2.2.2 Preprocessing

As preparation for the project, I sorted the image files by their creation time given by the metadata. The first 100 images for each class were visually inspected and images with identical YSTs and different camera settings were removed manually. These were

leftovers of early experiments with the camera setup. I removed further duplicates (identical images) by comparing file checksums. Images with labels that indicated no insects or unexpected insects not belonging to the class of the subset were also removed.

For the thrips, an additional visual inspection on this subset was carried out. This led to the exclusion of 200 images with wrong labels or too much other objects. To improve the quality of the thrips labels further, for the remaining pictures the bounding boxes given by the labels were cut out and inspected by an expert from the JKI. 1276 labels were identified as incorrect and removed from the label files.

Table 3 shows the number of images and labels after all cleaning reported in this section. The dataset is imbalanced, both in terms of the number of labeled images per class and the mean number of instances per image.

Class	Images	Labeled	Fraction labeled	Instances labeled	Mean instances per image
FungusGnats	1328	1018	76.66 %	33373	32.78
LeafMinerFlies	1422	659	46.34 %	6518	9.89
Thrips	1134	392	34.57 %	5453	13.91
WhiteFlies	1105	433	39.19 %	25735	59.43
Total	4989	2502	50.15 %	71527	28.59

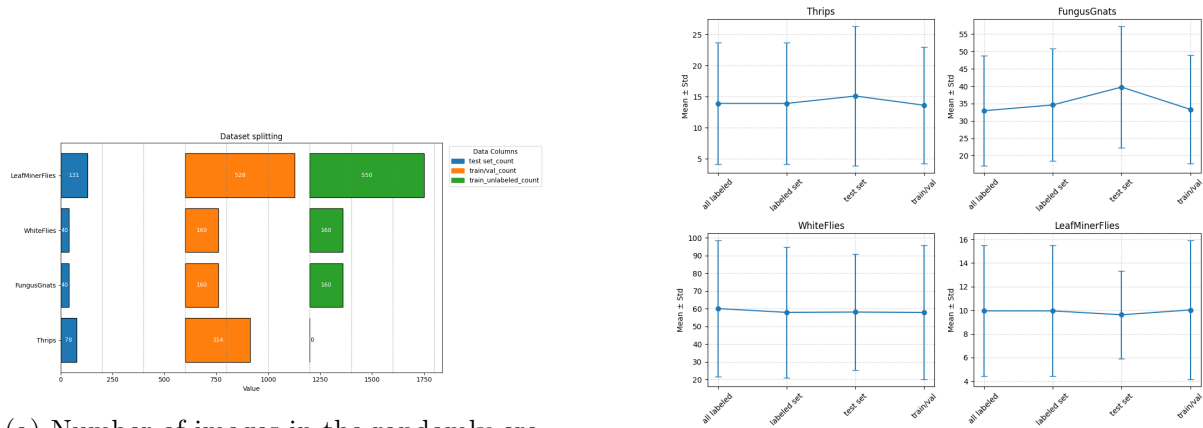
Table 3: Annotation statistics after cleaning and preprocessing.

Because of the summed difficulties the thrips images and labels posed, the class had to be discarded for all experiments but the supervised training, which showed that no satisfying results are to be expected in the proposed self-training. There are three major reasons for this decision: low sample size, presence of non-target objects, and low label quality. Because of size and morphological differences between adult males and females the thrips is the hardest pest class to detect and has to be represented with an according number of images and instances in the dataset. Unfortunately, after cleaning the thrips subset has the lowest number of images and labels in the dataset. Furthermore the thrips are easiest to be mixed up with other objects like foliage and therefore require clean images. But the thrips images are the only pest class that has lots of non-target objects, including thrips larvae on their images. The 1276 false labels clearly show that the automated labeling cannot work under the assumption that all objects on an image are instances of the respective pest classes. Finally, the inconsistent labeling quality prohibits proper testing.

Some labels were found to contain negative coordinates. After visual inspection of samples, these values were interpreted as their absolute counterparts, since this matched the actual object positions. The underlying cause of the negative coordinates could not be identified.

2.2.3 Splitting

From the dataset, three sets are built (see Figure 7a). **Short version:** From all labeled files, a subset was chosen. From this subset a test set and a train/val set was chosen (all random). Also a set without labels built.



(a) Number of images in the randomly created subsets. Sets are balanced because the absolute number of species per images is different for each class.

(b) Mean instances per image for the labeled (sub)sets. The differences between sets are minimal.

Figure 7: Data splitting

To balance the labeled sets, only 200 randomly chosen labeled images from the fungus gnats and the white flies are used, because these pest classes have considerably more instances on a single image than thrips and leaf miner flies. The other labels are discarded and the images retained as unlabeled. The labeled set is then further divided randomly into 20% test set and 80% training set. As Figure 7b shows, the mean instances per image are comparable for the full labeled set, the reduced labeled set, the test set, and the training set. From the unlabeled images excluding the thrips a set is drawn with comparable sizes as the training set. Finally, both training sets are further split into training and validation subsets, using the same 80/20 ratio.

2.3 Evaluation

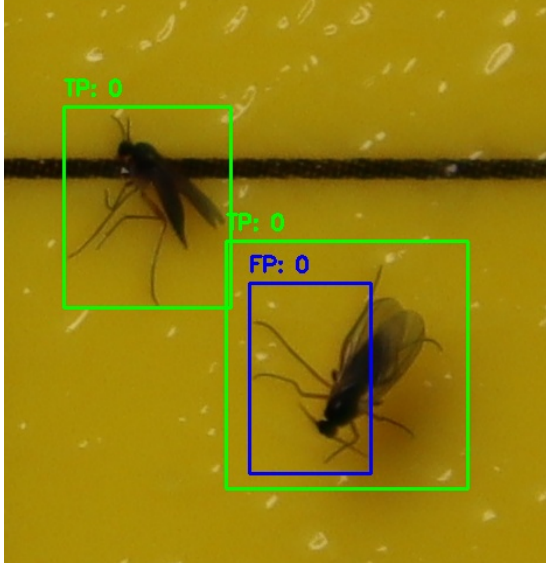
This section should contain:

- the sliding window prediction + nms + containment but without confidence threshold to get all predictions
- testing of stride 420/440 and containment filter on/off (see results, for stride include images from `cpu_vs_gpu.ipynb`)

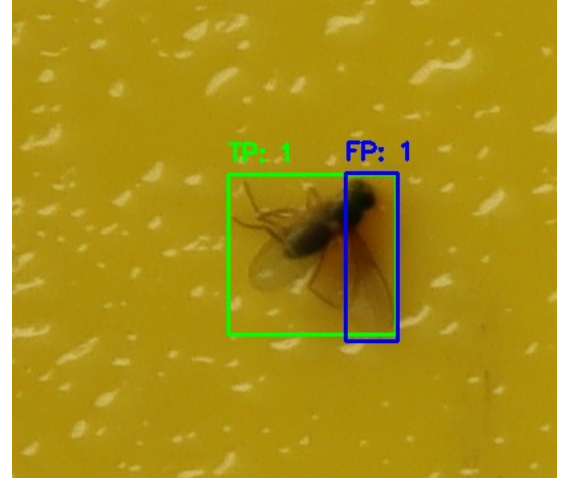
- comparing the boxes to the ground truth with thresholds for iou and another containment filter, greedy matching to allow only one true positive per GT box
- testing containment off/0.9/0.5 (see results)
- building of the prec/recall curve, find best thresholds by: a) if prec/recall arrive target values: take best f1 or b) take prec/recall with lowest distance
- sliding window prediction with per class confidence threshold applied to predict boxes
- for the evaluation of the image processing only the comparing to GT is needed
- best model found by comparing a) number of classes that reached target metrics, b) distance to target metrics area, c) f1 values
- filters build on visual inspections of the false positives/negatives and illustrated here with example images
- include a flowchart

Non-maximum suppression (NMS) is used to filter overlapping bounding boxes for the same object. It relies on the intersection of union (IoU) metric, which is the ratio between the area where two boxes overlap and the area that the boxes cover when combined. When two boxes have an IoU above a threshold, only the box with the higher confidence is kept. However, if a small box is completely or partially inside a larger box, the IoU will be low, because the intersection is only as small as the small box and the union is as big as the large box. As visual inspection shows, the small objects in the dataset make it necessary to include an extra filtering for this case. IoU for NMS was set to 0.4 and the threshold to filter out the other case was set to 0.5. See Figure 8 a and b for false positives that occur without the extra filter and see Table ?? that the extra filter reduces the number of false positives.

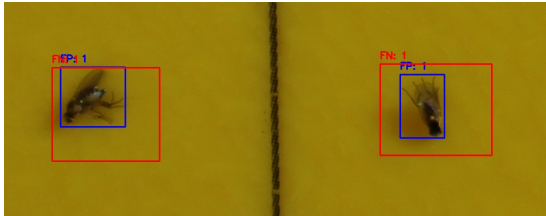
Comparison to the ground truth also relies on IoU and as usual, predicted bounding boxes were counted as true positives with an IoU equal or larger than 0.5. Again, visual inspection shows small correct predictions inside larger ground truth boxes are not covered by this procedure. In an additional routine, also predicted boxes contained at least with 0.5 of their area in ground truth boxes are counted as true positive. See Figure 8 c and d for false positives that occur without the extra comparison and see Table ?? that the extra comparison reduces the number of false negatives.



(a) Overlapping Bounding boxes not detected by NMS (fungus gnats), which creates a false positive box.



(b) Overlapping Bounding boxes not detected by NMS (leaf miner fly), which creates a false positive box.



(c) Bounding boxes counted as FP because of low IoU with the larger ground truth box (leaf miner flies).



(d) Bounding boxes counted as FP because of low IoU with the larger ground truth box (whitefly).

Figure 8: Detection errors because of low Intersection over Union (IoU) with small boxes. Green boxes are true positives (TP), blue boxes false positives (FP), red boxes false negatives (FN).

2.4 Supervised training

Short version: I used a YOLOv8 model. I trained on the full images resized to 1024 internally by YOLO and by two different sets of 640x640 segments built with stride 440. The sets differ by the inclusion strategy: The first set simply includes all resulting tiles. In the second tiling approach, for each object only one tile was allowed to avoid redundancy

in the training set and tiles with partial objects were dropped. Additionally for the first set the proportion of empty background images included and the exclusion of the thrips were tested. Augmentation was applied only carefully.

As a reference point for the automated labeling process, a YOLOv8 model is trained using the manual labels provided. The aim is to demonstrate that the data is sufficient to reach the target metrics of a minimum per-class precision of 0.9 and recall of 0.8. Since supervised training is not the main focus of this project, experiments are not designed to fully optimize results. A small YOLOv8s model pretrained on COCO is chosen because initial tests showed that larger models or later versions did not improve performance. With 4-5 GB of weights, the small model also meets the requirement of single-GPU training and the more modular code of version 8 is easier to adapt for self-training. No task-specific pretrained models are available to my knowledge.

The images are cropped to the size of the YST with color threshold based segmentation (compare YSTs in Figure 1 and Figure 4). This decreases the size of the dataset by 17,1 GB but also introduces different image sizes. Labels are therefore recalculated after cropping. Since YOLO labels are relative to image width and height, the new image dimensions must be used when transforming labels back into absolute coordinates.

YOLO expects images of size 640x640 and resizes images with larger resolution. Because the insect pest classes are of small size, this resizing might cause difficulties. Therefore images are tiled. After padding each image such that its dimensions are divisible by 640, the images are tiled into 640×640 patches with a stride of 440. The overlap of 200 pixels corresponds to the size of the largest pest class, the fungus gnats, such that an instance is at least contained in one tile. The labels are recalculated according to the tiles and labels and are only kept if they are contained to at least 80% inside the respective tile. The resulting tiles contain a large number of empty background tiles, which account for over 68% of the tiles in the labeled training set.

Starting from the default settings, short training runs with selected parameter changes were conducted to identify optimization potential, where augmentation settings and the ratio of background to foreground images are identified as the main factors influencing performance. Changing the default optimizer (stochastic gradient descent), learning rate schedule, and loss weights did not yield improvements. Because the dataset already contains considerable variance (see Section 2.2.1), augmentations are applied only subtly. The scaling factor is reduced to avoid unrealistic size differences between objects, and the probability of mosaic augmentation is lowered to prevent the model from learning lighting differences between images. A small amount of mixup is introduced with a 5% probability. All other parameters are kept at the default values.

All models were trained up to 200, then the best epoch (in terms of mAP@90:95) was taken.

Training run	Dataset	Background ratio	Best epoch
train1	tile v1	68%	144
train2	tile v1	30%	127
train3	tile v1 without thrips	47%	136
train4	full images	0 %	94
train5	tile v2	10%	181

Table 4: Selected training runs for supervised training

For the training runs the proportion of background tiles (tiles without objects) is varied (see Table 4). The initial training contains 68% background tiles, which is reduced to 10% and 30% in a number of training runs. To test the effect of the thrips images on the training of the other classes, the set with only 30% background images is stripped from all images from the thrips subset which changed the background to foreground ratio to 47%.

2.5 Image processing

Here I tried to summarize the basic ideas of the complete section, so this might be worth reading: The aim of the image processing is to detect a proportion of objects and create labels including bounding box coordinates, which will serve as the starting point for semi-supervised training of a YOLO model. The simple main idea of this step is to separate background and foreground with a color threshold and then keep all coherent foreground contours as objects. A rectangle around the contour serves as bounding box and the class id can simply be derived from the subset of the image, as all insects on an image are of the same pest class. For images without background structures the process reduces to the short and fast procedure reported in section 2.5.2. However, when images contain background structures—as in the dataset used in this study—more complex preprocessing is required, as detailed in Section 2.5.1. The presented pipeline was developed with the labeled training set. In between evaluation is done on the ground truth given by the manual labels. Raising the absolute number of true positives and minimizing the number of false positives are the main metrics. Precision is also tracked but is considered secondary, as it might come at the cost of too less true positives. Final evaluation was done on the test set.

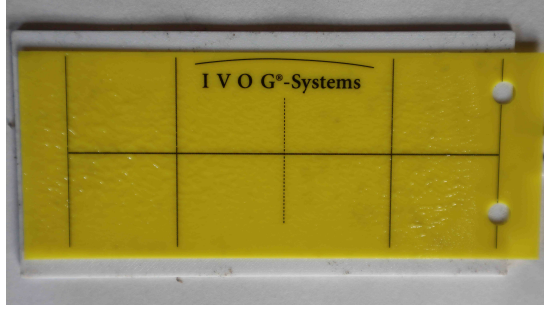
2.5.1 Preprocessing for segmentation

Short version: I created some masks (Figure 9) and devised a processing pipeline to apply them (schematic is now only a bullet point list, illustrations Figure 10). The result is a binarized image which divides foreground and background.

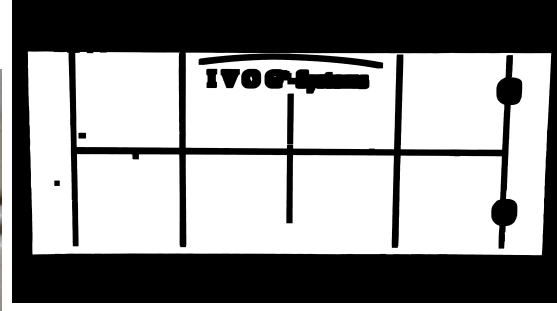
To filter out background structures, a mask covering these regions is applied to each image. The pixels within the mask are changed to a yellow hue, allowing them to be identified as background during segmentation. Direct detection of the lines was tried, but led to no useable results, as the detected line segments were often very short and could not be separated from the line segments of the objects, that should be kept. Because the images in the dataset were captured from slightly different angles, the mask must be aligned individually for each image. All image processing was performed using the cv2 Python library. Binarization, which is used at several steps, is achieved through color thresholding in the HSV color space, as the hue component provides robustness against variations in lighting. The thresholds used were: Hue 20–30, Saturation 100–255, and Value 100–255.

The processing pipeline begins with a subset of uncropped images, as determined by the splitting described in Section 2.2.3. Since the masking procedure assumes the header is located at the top of the YST, images were rotated when necessary. The required rotation was detected by dividing the YST area into horizontal halves and comparing the summed black pixel area in each half, with black pixels defined as having a value below 50. This method becomes unreliable when the YST is crowded with a large number of black insects, so a visual inspection was performed to ensure accuracy. Correspondingly, the labels used for manual evaluation were rotated to match the corrected image orientation.

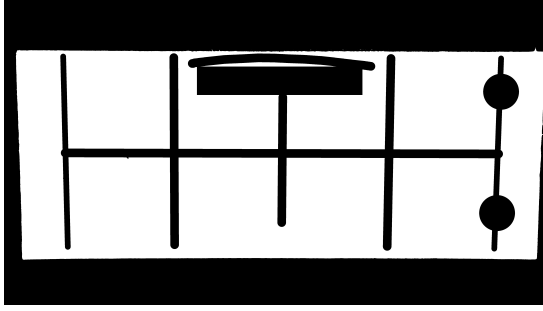
Creating the mask required balancing line thickness to fully cover background structures despite minor misalignments, while avoiding excessive coverage of the objects. Binarized and denoised images of empty YSTs, captured under the same conditions as the dataset images, were used for mask generation. Two approaches were employed: a handcrafted mask, where background structures were manually overlaid with black lines (thickness 63 and 96 pixels) in GIMP (Figure 9c), and an image-processing-based mask, created by combining several empty YST images with projective transformations and thickening the black structures via dilation (kernel size 25) (Figure 9b). From this mask, three additional variations were generated by applying further dilation (kernel size 50) (Figure 9d), manually thickening the right border lines to cover the holes completely (Figure 9e), and drawing a rectangle over the header (Figure 9f). For subsequent alignment, the corners of the YST contour and two points of the horizontal midline calculated from the original masks were saved.



(a) Image used for the creation of masks.



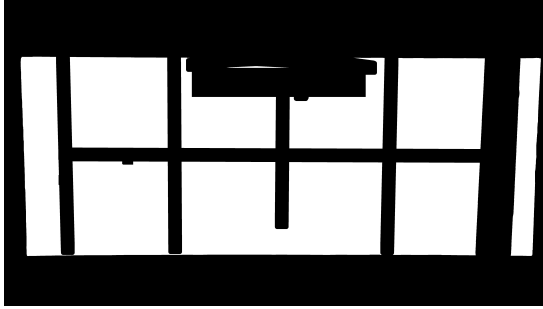
(b) Mask created by overlaying other images to the initial one and then thickening the lines.



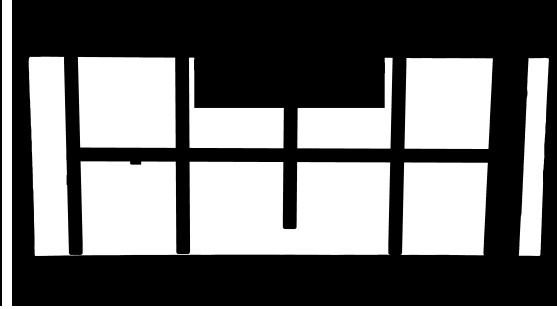
(c) Mask created with a graphics program (GIMP).



(d) Mask from b) with thicker lines.



(e) Mask from d) that covers the holes.



(f) Mask from e) with a rectangle as header.

Figure 9: Masks used for masking out the background structures.

With the masks prepared, preprocessed and masked versions of the image set were created. The pipeline of this process is divided into the following steps.

- Find contours on the binarized image. Take the largest, which is the YST. (algorithm used: `cv2.findContours`)
- Assuming a rectangle shape find the corners or exclude the image from further processing. (see the corners in Figure 10a, algorithms used: `cv2.arcLength`, `cv2.approxPolyDP`)
- Using the corners of the mask and the image, do a projective transformation of the mask and of the midline of the mask. (see Figure 10b, algorithms used: `cv2.findHomography`, `cv2.warpPerspective`)

- Using the middlepoints of middle horizontal lines of the mask and the image, do another affine transformation of the mask, if the difference is lower than 500 pixels. If the result is larger, a wrong horizontal line was picked and the transformation is skipped. (see Figure 10c, algorithms used: cv2.HoughLines, cv2.perspectiveTransform, cv2.warpAffine)
- Color the pixels given by the aligned mask in the image yellow (background) as seen in image Figure 10.
- Crop the image to the YST (and recalculate the manual labels for evaluation).

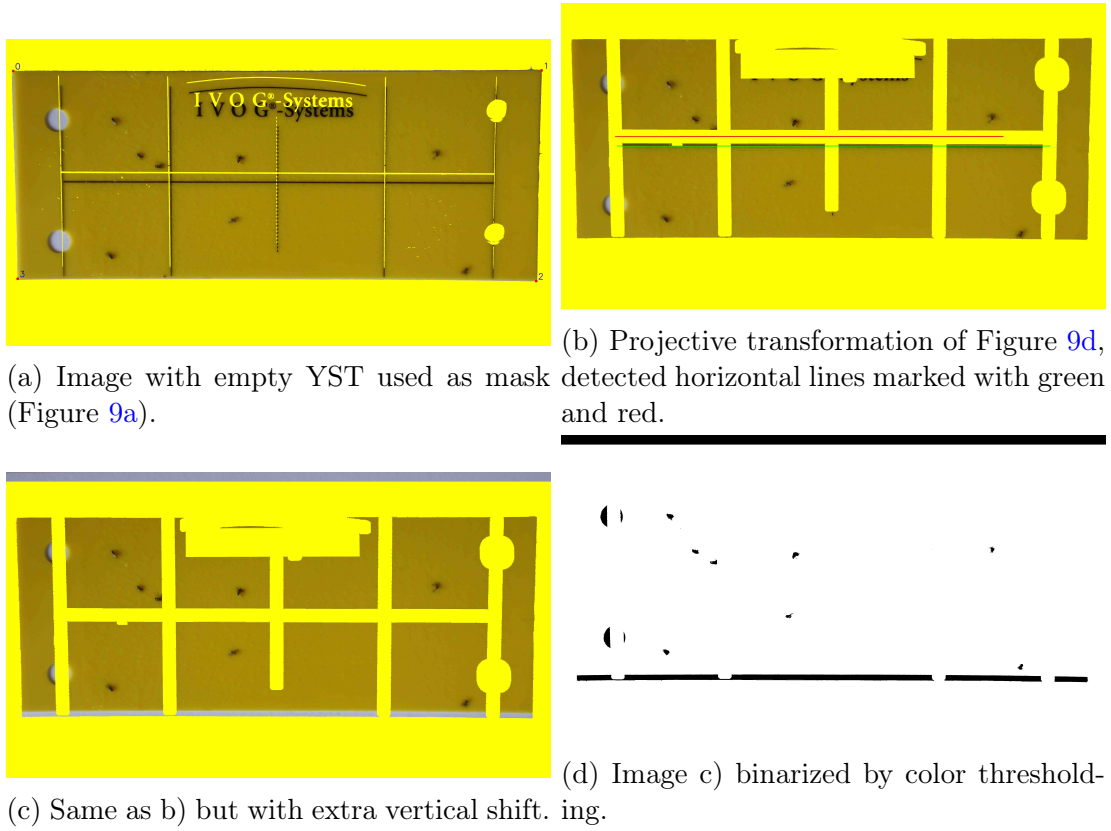


Figure 10: Steps for creating binarized images.

2.5.2 Segmentation

Short version: The foreground objects need to be filtered further. For testing the mask above this was done by simply thresholding the size of the foreground contours. For final use a more refined set of thresholds is proposed that includes statistically derived thresholds for the size and the shape of the rectangles created around the contours. (see Table 5 and Figure 11 for illustrations).

The segmentation is done on the binarized, preprocessed images as seen in Figure 10d. The results of the segmentation are filtered by several thresholds. A first set of threshold is used to evaluate the effects of the different masks as seen in Figure 9. A second, more refined set of thresholds is derived from inspecting the data given by the best evaluated mask (Figure 9e. Evaluating was done on the manual labels of the labeled training set. Comparing to GT was reduced to IOP, because it gives nearly identical results for smaller predictions inside ground truth and is more strict for overlapping ones.)

The first set entails guessed lower and upper threshold for the area contours detected. For fungus gnats and leaf miner flies only contours with an area in the range between 1000 and 10000 pixels are kept, for the whiteflies the range is between 100 and 1000 pixels. The rectangles around the contours kept are then scale with factor 1.5, because the contours lost small features like legs, that would cause the rectangles to be too small (see Figure 11. A second threshold then filters the rectangles by the ratio of their height and width. Assuming that more oblong rectangles are likely to be non-insect objects (e.g. left over background elements, foliage, parts of insects), the maximum ratio was set to 2. This excludes rectangles with one side being more than twice as long as the other side.

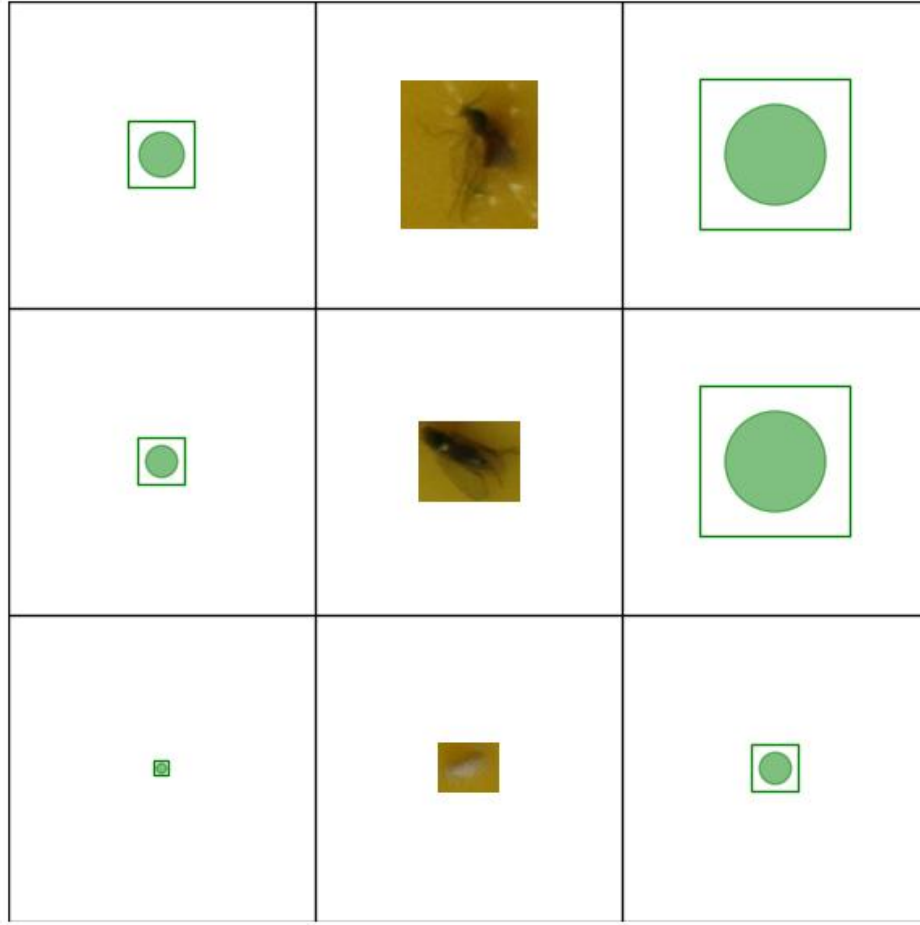


Figure 11: Size comparison, from top to bottom: Fungus gnats, Leaf miner flies, Whiteflies. Green circles are the minimum and maximum contour area allowed. Green rectangles are the resultant rectangles for the areas, which are squares in this case.

For each mask, segmentation is done for all preprocessed images and the results were evaluated by comparing to the manual ground truth with the Intersection over Prediction metric with threshold 0.5. The results from the best performing mask Figure 9e are used to inspect the distribution of box sizes, box ratios and, only for the whiteflies, the value in the HSV-space for all rectangles. Additional filter thresholds were then tested on the calculated rectangles that are given in Table 5. Goals were to reduce the number of small insect parts and occasional non-insect objects, as well as the number of large boxes with more than one insect occluding each other. Only for the whiteflies a considerable number of overlapping boxes were created, presumable because individual objects often led to two separate contours. Therefore this case was checked and only one rectangle kept.

Species	Threshold	Filter set 1	Filter set 2
Fungus gnats	Min contour	1000	2000
	Max contour	10000	<i>same</i>
	Max box size	-	95th percentile
	Max box ratio	-	95th percentile
	Min value	-	-
Leaf miner flies	Min contour	1000	<i>same</i>
	Max contour	10000	<i>same</i>
	Max box size	-	95th percentile
	Max box ratio	-	95th percentile
	Min value	-	-
Whiteflies	Min contour	100	<i>same</i>
	Max contour	1000	<i>same</i>
	Max box size	-	95th percentile
	Max box ratio	-	95th percentile
	Min value	-	5th percentile

Table 5: Thresholds of the two filter sets for filtering the results of the segmentation.

2.5.3 Augmenting the data

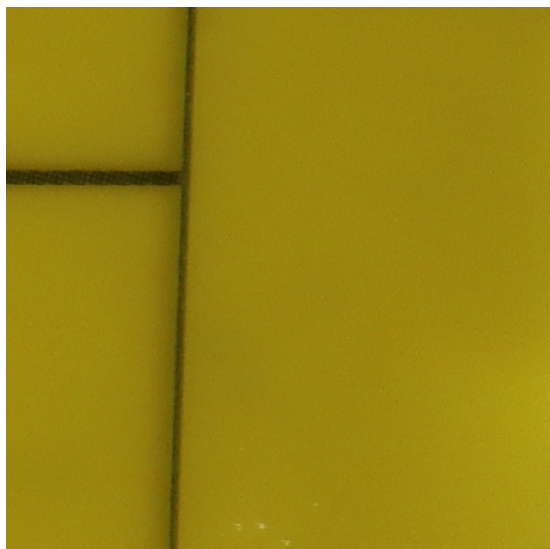
Masking the black background structures creates a biased label set and the model learns to ignore insects on background structures. As a remedy, cut out insect will be pasted onto empty YSTs to create artificial data with correct labels (see Figure below for an example). The results can be used as additional training data.



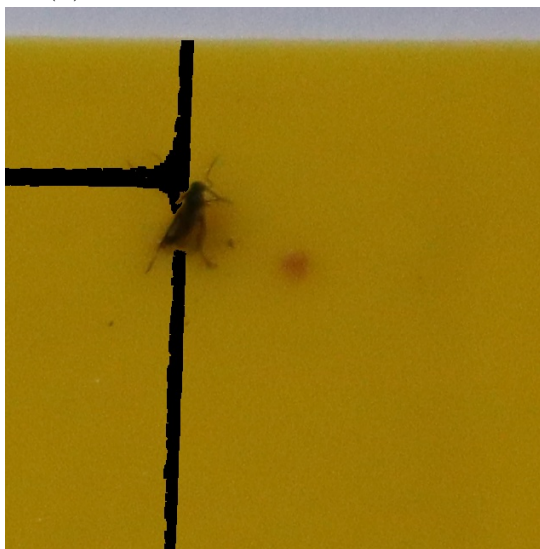
(a) Empty YST with some leaf miner flies pasted.



(b) Original tile as used for training.



(c) Empty background tile with background structures.



(d) Merged tiles usable as new training data.

Figure 12: Examples for augmented training data.

2.6 Self-Training

Short version: I trained on the labels created by image processing and tested a bunch of things.

2.6.1 Training on the labels created by image processing

- was done on tiles (as explicated in the section on supervised training)
- only tiles including at least one labeled instance were included in the train set

- for validation subset the manual labels are kept
- augmentations were taken from above (filter set 2), but also rotation was tested
- tests on epoch lengths
- tests on inclusion of empty tiles
- also compared to the results on the unlabeled set
- will also be tested with artificially populated empty YSTs

2.6.2 Further training on predicted labels

- as of now this is not included in the results, because no improvements were made
- new labels were included as pseudo-labels filtered by a confidence threshold
- predicted class was hardcoded from the species given by the filename/datasubset
- epoch length 5 & 10 was tested
- inclusion of new images with new predictions was tested (on unprocessed images)
- weighing of the losses for the pseudo-labels (with disabled mosaic/mixup augmentations) was tested
- for later iterations the addition of tiles with no predictions at all as empty tiles was tested

3 Results

3.1 Comparing supervised training and self-training

I would first compare the final result of the best supervised model and the best model trained on the image processed labels side by side. As of now, both do not meet the target metrics.

3.2 Evaluation settings

Topic sentence: The additional filtering of the predicted boxes and the additional checking for contained boxes in the comparison to the ground truth allows better detection results.

run	Stride	Containment filter	Containment comparison	TP	FP	FN
train1	440	-	-	3240	637	1024
train1	440	0.5	-	3240	592	1027
train1	440	0.5	0.9	3331	501	936
train1	420	0.5	0.9	3569	540	698
train1	440	0.5	0.5	3601	508	666
train1	420	0.5	0.5	3601	508	666
train2	440	-	-	3469	664	798
train2	440	0.5	-	3469	664	798
train2	440	0.5	0.9	3469	584	798
train2	420	0.5	0.9	3737	716	539
train2	440	0.5	0.5	3469	587	798
train2	420	0.5	0.5	3737	637	530

Table 6: Comparison of detection results under different settings of extra thresholds. The models are trained each 100 epochs, train1 on the full trained label set, train2 with only 30% of the empty background tiles. Confidence threshold is set to 0.5, non-maximum suppression threshold for filtering before the ground truth to 0.4 and IoU threshold for comparison to ground truth to 0.5 in all tests. Changing the stride to 420 always brings a boost for the number of true positives, the extra containment filter before comparing the boxes brings a slight reduction in false negatives only for train1. The extra containment section in the comparing to the ground truth with 0.9 always brings benefits in all three metrics, but setting the threshold to 0.5 only improves results for stride 420.

3.3 Supervised training

Topic sentences:

- Training on the full images let the larger insect pest classes arrive the target metrics consistently. The two smaller classes either missed one (whiteflies) or both targets.
- Training on the full tile sets let only the leaf miner flies arrive the target metrics consistently. Fungus gnats and also whiteflies came close. The small pest classes profited from the tiling approach.

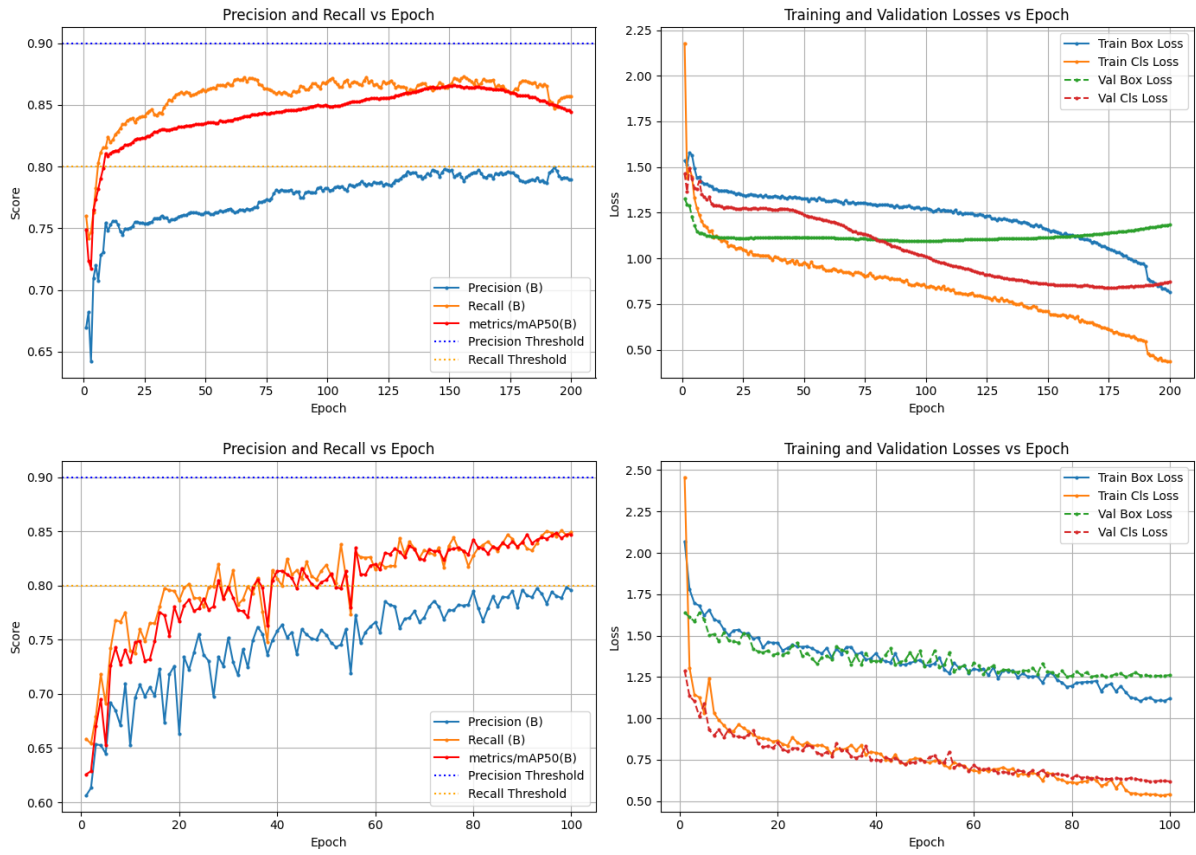
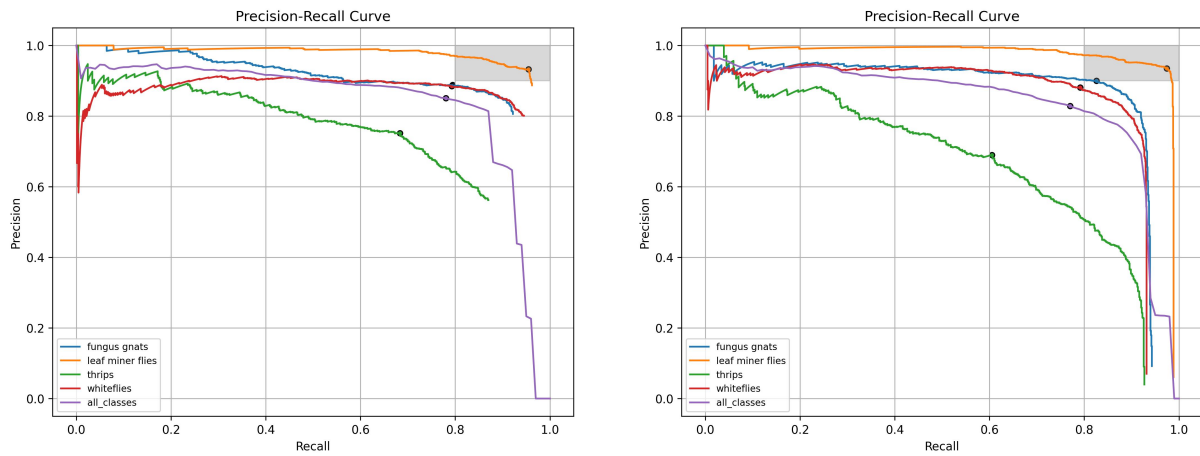


Figure 13: Training curves for train1 (on full tiled set) and train4 (on full images) as seen in Table 4. Longer training brings no benefits. The smoothness of the first curve shows how the tiling reduced the noise? But not the problematic noise, as results are not better?



(a) Train1 – tiled set

(b) Train4 – full images

Figure 14: Precision–recall curves for train1 (on full tiled set) and train4 (on full images) as seen in Table 4. Target metrics are marked as shaded boxes. Training on full images improves results for larger pest classes slightly, while training on tiles is better for smaller objects.

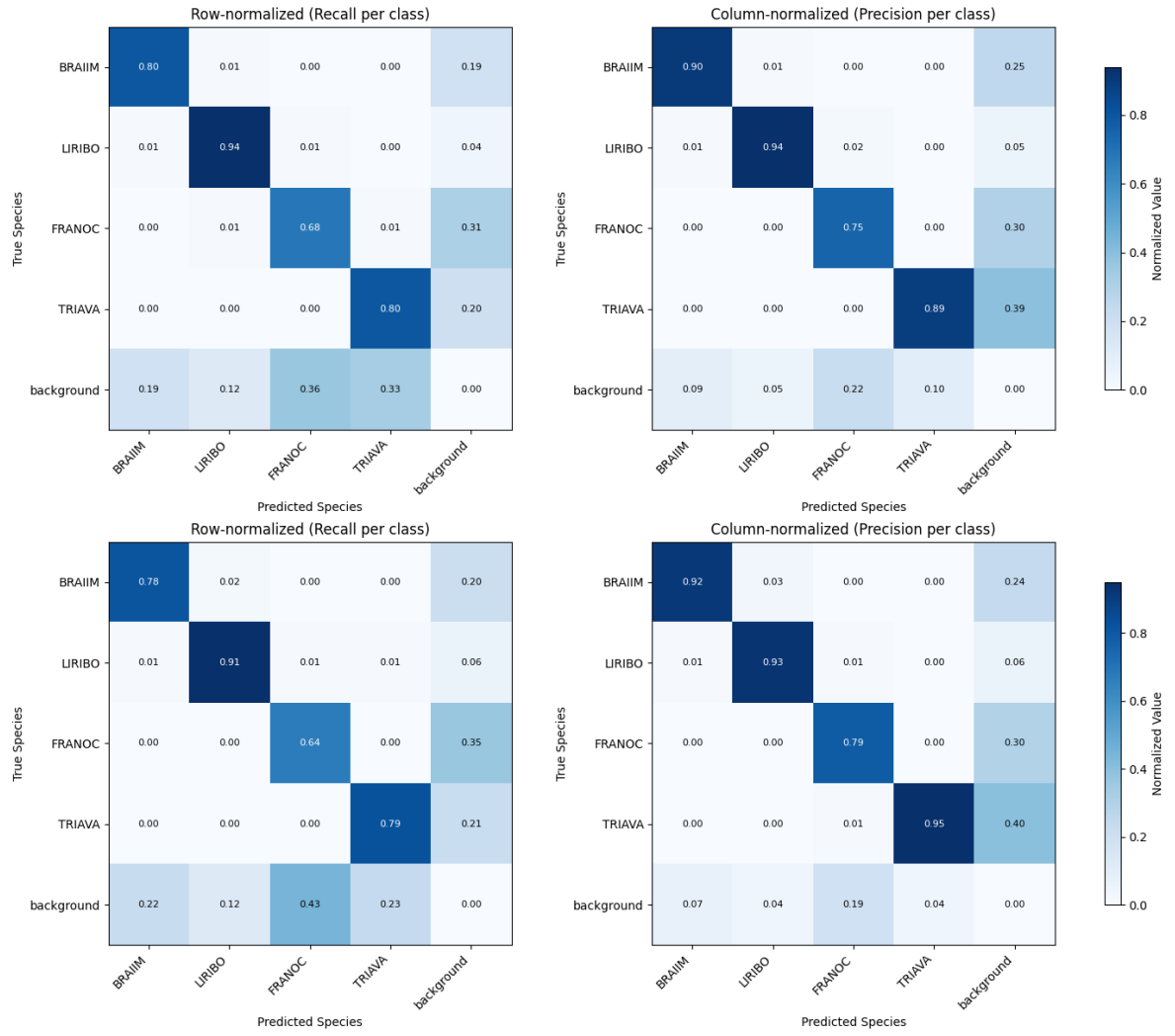


Figure 15: Top/down: Confusion matrices for train1 (on full tiled set) on validation and test set. The results are consistent.

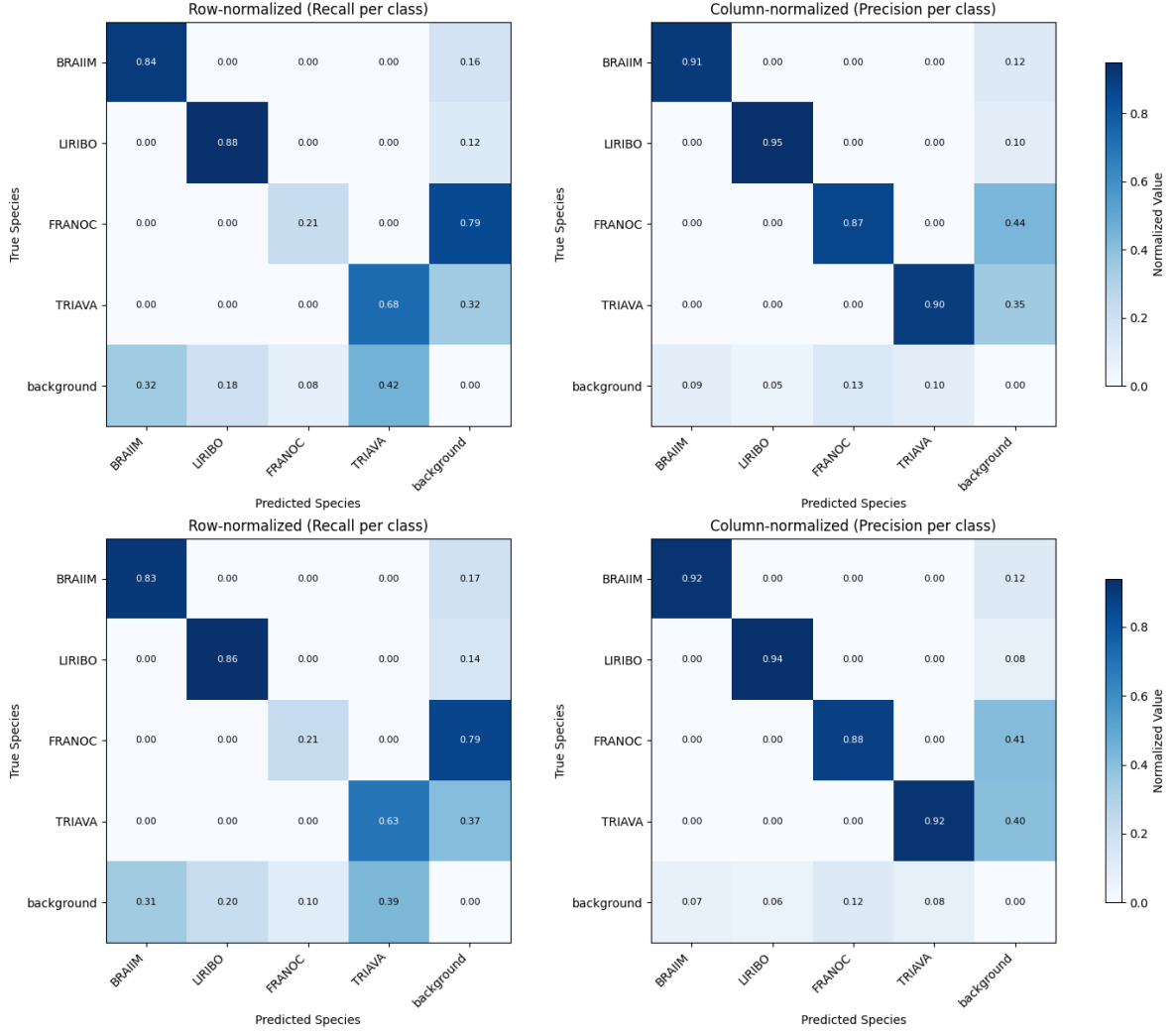


Figure 16: Top/down: Confusion matrices for train4 (on full images) on validation and test set. The results are consistent.

3.4 Image processing

The first subsection gives the results for testing the different masks that are needed to filter out background structures. In the second subsection the results for the different sets of thresholds that are needed to filter the rectangles created by the image processing are reported.

3.4.1 Preprocessing for segmentation

Topic sentence: The masks are used to filter out the background structures from the image before segmenting it to find the insects. The best mask decreased the number of false positives while retaining a sufficient number of true positives for the self-training.

Figures&Tables: Results for the different masks and the application procedures can be compared in Figure 17. Thresholds used for creating the results are found in the methods section, Table 5, filter set 1. Using this filter set and the mask with the best results, statistics were taken and used as threshold for filter set 2 (see Table 7).

Figure 17 shows the results of applying the different masks, filtered with the first set of thresholds given in Table 5 and evaluated with the given manual labels. First the masks Figure 9a and Figure 9b are simply overlaid to the respective image without transformation and as the direct comparison to test 3 shows, the alignment of the mask improves the results overall. The same is true for the mask with thicker lines Figure 9d tested with alignment in test 4, albeit at the cost of reducing the count of the true positives. Further reworking of the masked Figure 9e and Figure 9f lead to slightly reducing false positives again at the cost of reducing true positives. Introducing the vertical shift for better alignment of the mask in the last test, again with Figure 9f reduces the false positives of the leaf miner flies considerably. The mask and the two transformations of the last test are used as basis for further experiments. **Exact results in (mask_testing_collected_results.csv)**

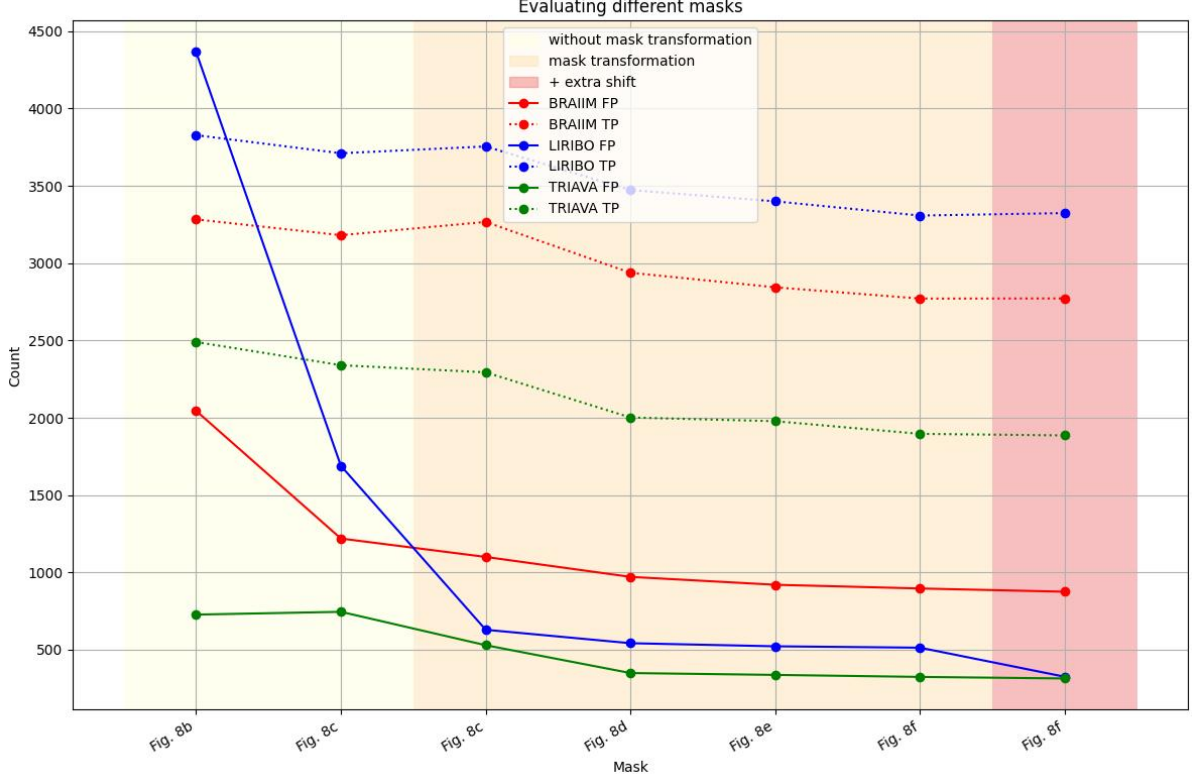


Figure 17: Evaluating the masks given in Figure 9. The thicker lines of mask 8c in combination with the transformation (i.e. adapting the mask to each single image) reduces false positives by a large margin. Even thicker lines tend to also decrease the true positives. The extra shift of the mask mainly reduces the number of false positives for the leaf miner flies.

Training Set	Metric	Fungus Gnats	Leaf Miner Flies	Whiteflies
Labeled Training Set	95th percentile box size	41704	22340	3168
	95th percentile box ratio	1.75	1.76	1.88
	5th percentile box value	-	-	97
Unlabeled Training Set	95th percentile box size	42970	28530	3477
	95th percentile box ratio	1.73	1.76	1.84
	5th percentile box value	-	-	96

Table 7: Found possible thresholds for labeled and unlabeled training set.

The mask with the best results (Figure 9f) produced distributions of boxes sizes, deviations from 1:1 ratio of sidelenghts and distribution of values for the whitefly rectangles, all taken from the results of the simple filter set 1 (see Table 5. Table 8 gives the found values for the metrics for both the labeled and the unlabeled training set, which are similar as expected.

3.4.2 Segmentation

Topic Sentence: Applying a refined filter set with thresholds derived by statistics reduced the number of false positives further and increased precision while retaining a sufficient number of true positives for the following self-training.

Figures&Tables: Figure 18 shows the results of applying the filters. Table 8 shows the resulting numbers of rectangles created by the image processing. Figure 19 shows the results of applying the image processing to the test set.

Figure 18 shows the results for the tests with the found thresholds. The two filter sets 1 and 2 as given in Table 5 are always the first and last steps in the graphs. In between single changes to set 1 are visualized. For the fungus gnats, all tried changes yield smaller effects by tolerable reductions of the number of true positives and were combined in the end. The number of false positives is still the highest compared to the other classes, which as already observed in the evaluation of the supervised training might be mainly due to missing labels. For the leaf miner flies, the effect are comparable but less prominent as in the fungus gnats, but precision and number of false positives were also already acceptable only with the first filter set. For the whiteflies, filtering the rectangles by box size and ratio was not taken over into the filter set, because it reduced the number of already scarce true positives too much. Instead overlapping boxes were deleted and filtering by value of the rectangles reduced false positives and at the same time retained the number of true positives.

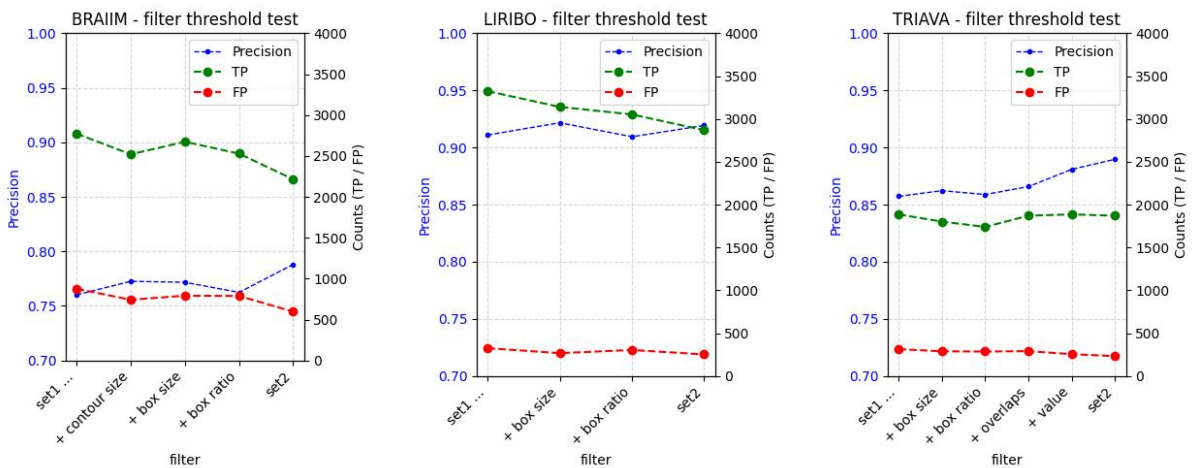


Figure 18: Results for different filters for each species. Leftmost is always the parameter set 1 with no extra filtering, rightmost is parameter set 2. In between single additions to set1 are tested. (Tested on lables of the labeled training set) Species do not follow my usual naming convention. Visual style is different from later figures with comparable content (two y-axes)

As to be expected objects of the same color, shape and size cannot be filtered out. This concerns mainly parts of the insects as well as malformed or very untypical instances, as well as insects occluding each other. Table 8 gives the absolute numbers of rectangles returned which each filter set for the labeled and unlabeled training set. As expected they turn out to be very similar with exception of the total numbers of whiteflies, which is doubled in the unlabeled training set. As the number of filtered out rectangles is also proportionally larger, this is possibly only a effect of more crowded whitely images in the unlabeled set.

Class	Labeled training set		Unlabeled training set	
	Filter set 1	Filter set 2	Filter set 1	Filter set 2
Fungus gnats	3647	2800	3477	2691
Leaf miner flies	3587	3297	3112	2714
Whiteflies	2200	2141	5010	4916
Total	9434	8238	11599	10321

Table 8: Absolute number of rectangles created **Why are the whitefly counts in the labeled training set not balanced?**

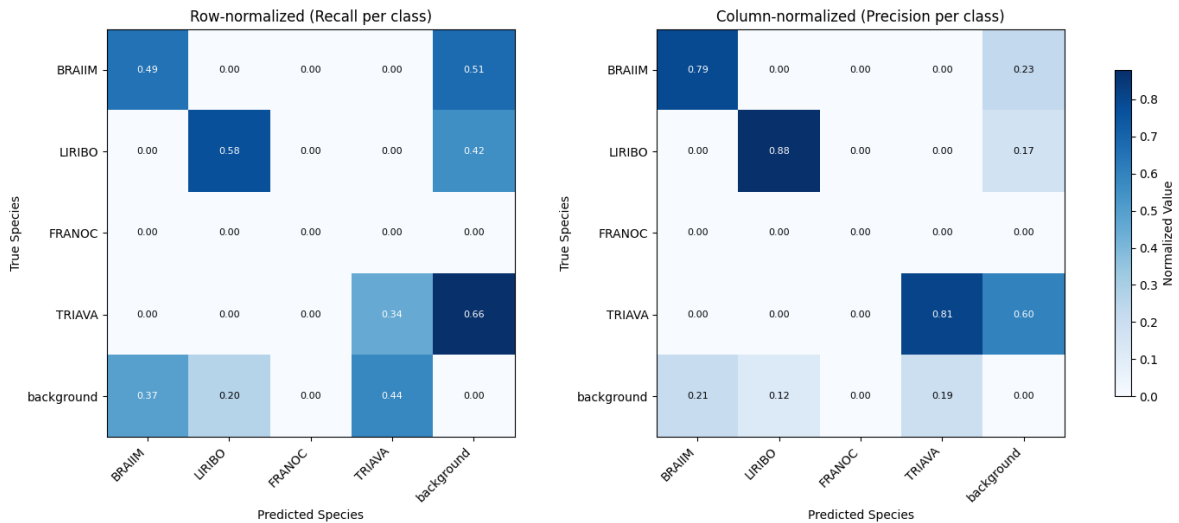


Figure 19: Comparing the image processed labels of the test set to the ground truth. Precision is about 0.1 below the target and is supposed to lead to good quality labels in training on these labels, recall is very low. **The confusion matrices actually show the result of a slightly improved image processing pipeline (only for the whiteflies though and without much improvement). This is not reported in the methods yet. Also I should maybe emphasize that all other results in this section are taken on the manual labels of the training set and I should recheck the evaluation process of these in comparison to the evaluation on the test set.**

3.5 Training on the labels created by image processing

Topic sentences: A model trained for 10 epochs on the image processed labels with empty background tiles included and rotation as augmentation enabled reached the best results.

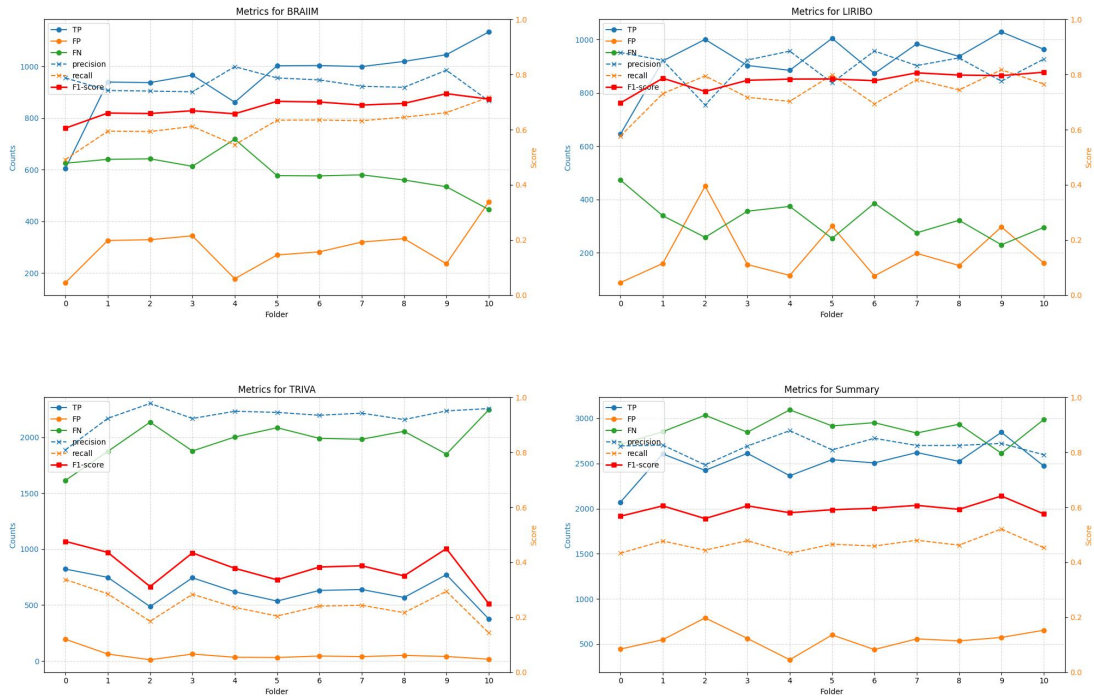


Figure 20: Results of initial image processed labels ("0") and different models trained on those labels (details in methods tba) tested on the test set (but should have been tested on the val set with correct conf threshold). Training run 9 combined lowest number of false negatives with highest number of true positives and has therefore the highest F1-score. Whiteflies (= TRIVA) do not profit from training.

The model learned to discriminate classes but also improved recall compared to the image processed labels.

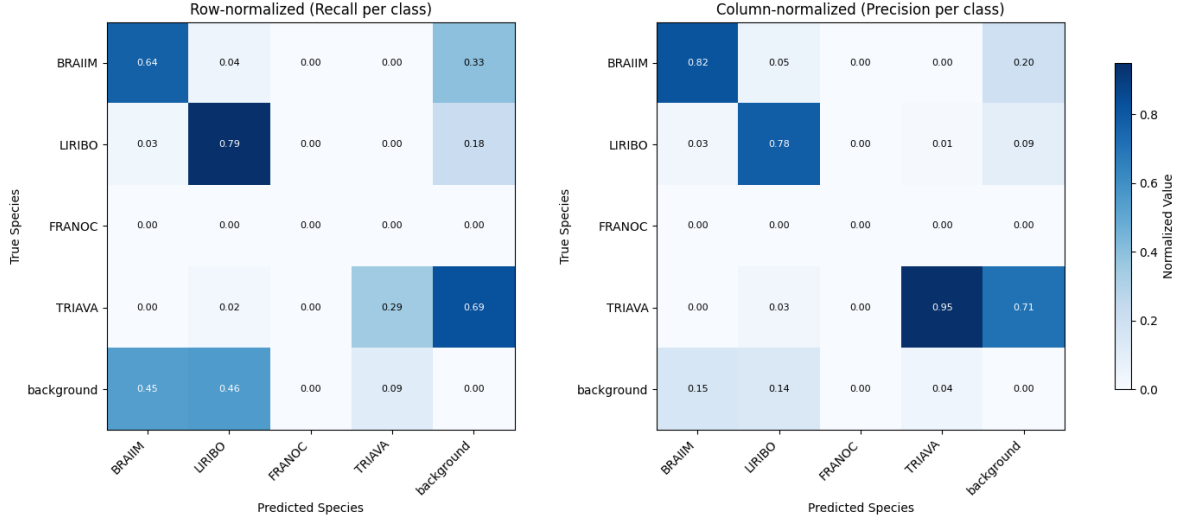


Figure 21: Comparing the predictions on the test set (should be val set with correct conf threshold.) of the model trained on the image processed labels to the ground truth. For leaf miner flies and fungus gnats recall increased, for whiteflies precision increased.

The model did not simply copy the implicit rules of the image processed labels.

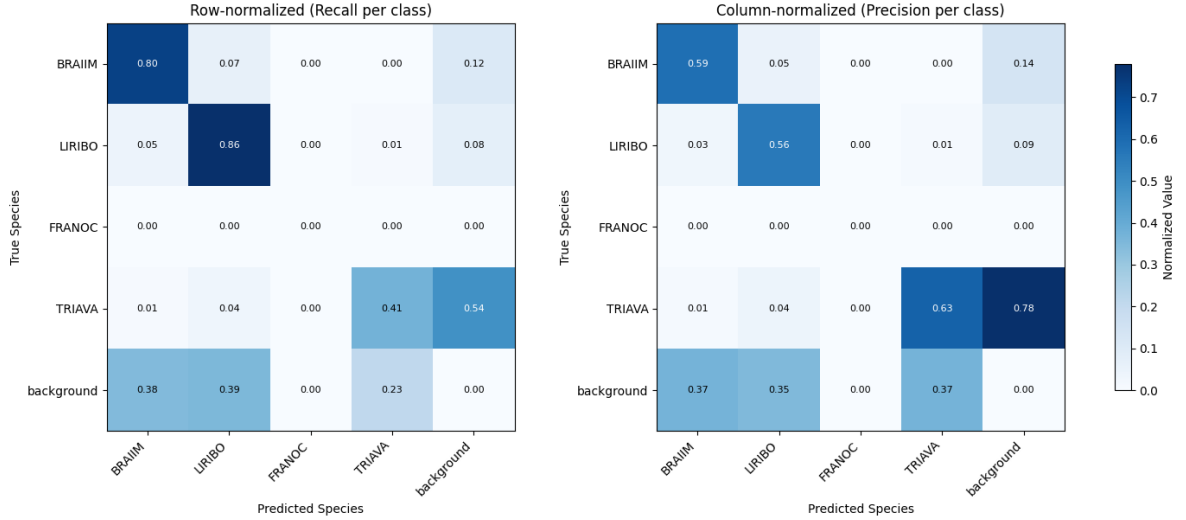


Figure 22: Comparing the predictions on the test set (should be val set with correct conf threshold.) of the model trained on the image processed labels on the image processed labels of the test set. This is probably not the most intuitive comparison. Maybe make a graph do visualize the idea behind it?. The model has lower precision on the image processed labels than on the ground truth (as seen above in Figure 21), which may be a sign of generalization.

At least 39% of the false positives are at least 50% inside the masked regions and also at least 34% of the false positives are located fully in that area - or in other words, if I would ignore false detections in the masked area the model would be much better. The model learns to reject insects on the background structures and also learns some

background structures as insects. I will try adding augmented image with insects placed on background structures to circumvent this.

4 Discussion

results supervised: why the thrips are hard to detect? because male adults and larvae are so similar and the labels are ambiguous

Data in a later implementation will differ in at least two aspects. Most notably a domain shift has to be expected, as images taken in real use cases will likely be of much lower resolution. add picture of the checkpoint? mention resolution of thrips, might be solvable by SSL, too Also, in the final usage, several pictures of the same YST are taken over the day. Because the individuals can interact with the glue and become partially hidden or change their appearance over time, this poses an additional problem to be addressed. If the difference between the individuals in the training data and those in the final data becomes too great, individuals might no longer be detected, and the population might be underestimated.

5 Conclusion

6 References

- [1] Curtis A. Deutsch et al. “Increase in crop losses to insect pests in a warming climate”. In: *Science* 361.6405 (2018), pp. 916–919. DOI: [10.1126/science.aat3466](https://doi.org/10.1126/science.aat3466).
- [2] Laure Mamy et al. *Impacts des produits phytopharmaceutiques sur la biodiversité et les services écosystémiques. Rapport de l’expertise scientifique collective*. Research Report. INRAE ; IFREMER, 2022, 1408 p. DOI: [10.17180/0gp2-cd65](https://doi.org/10.17180/0gp2-cd65).
- [3] Matthew R. Smith et al. “Pollinator Deficits, Food Consumption, and Consequences for Human Health: A Modeling Study”. In: *Environmental Health Perspectives* 130.12 (2022), p. 127003. DOI: [10.1289/EHP10947](https://doi.org/10.1289/EHP10947).
- [4] URL: <https://www.fao.org/pest-and-pesticide-management/ipm/integrated-pest-management/en/> (visited on 04/04/2025).

- [5] G.J. Messelink and H.M. Kruidhof. “Advances in pest and disease management in greenhouse cultivation”. In: *Achieving sustainable greenhouse cultivation*. Ed. by L. Marcelis and E. Heuvelink. United Kingdom: Burleigh Dodds Science Publishing, Sept. 2019, pp. 311–356. DOI: [10.19103/as.2019.0052.17](https://doi.org/10.19103/as.2019.0052.17).
- [6] URL: https://food.ec.europa.eu/plants/pesticides/sustainable-use-pesticides/integrated-pest-management-ipm_en (visited on 04/02/2025).
- [7] Tiago Domingues, Tomás Brandão, and João C. Ferreira. “Machine Learning for Detection and Prediction of Crop Diseases and Pests: A Comprehensive Survey”. In: *Agriculture* 12.9 (2022). DOI: [10.3390/agriculture12091350](https://doi.org/10.3390/agriculture12091350).
- [8] Ana Cláudia Teixeira et al. “A Systematic Review on Automatic Insect Detection Using Deep Learning”. In: *Agriculture* 13.3 (2023). DOI: [10.3390/agriculture13030713](https://doi.org/10.3390/agriculture13030713).
- [9] Mamta Mittal et al. “Machine learning for pest detection and infestation prediction: A comprehensive review”. In: *WIREs Data Mining and Knowledge Discovery* 14.5 (2024). DOI: <https://doi.org/10.1002/widm.1551>.
- [10] Joseph Redmon et al. “You only look once: Unified, real-time object detection”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 779–788. DOI: [10.48550/arXiv.1506.02640](https://doi.org/10.48550/arXiv.1506.02640).
- [11] Dan Jeric Arcega Rustia et al. “Automatic greenhouse insect pest detection and recognition based on a cascaded deep learning classification method”. In: *Journal of Applied Entomology* 145.3 (Apr. 2021), pp. 206–222. DOI: [10.1111/jen.12834](https://doi.org/10.1111/jen.12834).
- [12] Wenyong Li et al. “Field detection of tiny pests from sticky trap images using deep learning in agricultural greenhouse”. In: *Computers and Electronics in Agriculture* 183 (Apr. 2021), p. 106048. DOI: [10.1016/j.compag.2021.106048](https://doi.org/10.1016/j.compag.2021.106048).
- [13] Dinis Costa et al. “Intelligent System for Automatic Whitefly Detection in Tomato Greenhouses”. In: *2023 6th Experiment@ International Conference (exp.at'23)*. Évora, Portugal: IEEE, June 2023, pp. 29–30. DOI: [10.1109/exp.at2358782.2023.10546195](https://doi.org/10.1109/exp.at2358782.2023.10546195).
- [14] Xiaolei Zhang et al. “Automatic pest identification system in the greenhouse based on deep learning and machine vision”. In: *Frontiers in Plant Science* 14 (Sept. 2023), p. 1255719. DOI: [10.3389/fpls.2023.1255719](https://doi.org/10.3389/fpls.2023.1255719).
- [15] Song Wang et al. “A Deep-Learning-Based Detection Method for Small Target Tomato Pests in Insect Traps”. In: *Agronomy* 14.12 (Dec. 2024). Publisher: MDPI AG, p. 2887. DOI: [10.3390/agronomy14122887](https://doi.org/10.3390/agronomy14122887).

- [16] Jianyu Shi et al. “Small Target Insect Detection Based on Improved YOLOv8n”. In: *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Hyderabad, India: IEEE, Apr. 2025, pp. 1–5. DOI: [10.1109/icassp49660.2025.10890801](https://doi.org/10.1109/icassp49660.2025.10890801).
- [17] Dan Jeric Arcega Rustia et al. “Online semi-supervised learning applied to an automated insect pest monitoring system”. In: *Biosystems Engineering* 208 (Aug. 2021), pp. 28–44. DOI: [10.1016/j.biosystemseng.2021.05.006](https://doi.org/10.1016/j.biosystemseng.2021.05.006).
- [18] Xianwei Li et al. “MATeacher: Multi-scale Views Learning and Adaptive Unsupervised Weight for Semi-supervised Pest Region Detection”. In: *2024 6th International Conference on Robotics, Intelligent Control and Artificial Intelligence (RICAI)*. Nanjing, China: IEEE, Dec. 2024, pp. 698–703. DOI: [10.1109/RICAI64321.2024.10911445](https://doi.org/10.1109/RICAI64321.2024.10911445).
- [19] Paweł Majewski et al. “Improved Pest Detection in Insect Larvae Rearing with Pseudo-Labeling and Spatio-Temporal Masking:” in: *Proceedings of the 19th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*. Rome, Italy: SCITEPRESS - Science and Technology Publications, 2024, pp. 349–356. DOI: [10.5220/0012311300003660](https://doi.org/10.5220/0012311300003660).
- [20] Jiale Zhou et al. “Mutual learning with memory for semi-supervised pest detection”. In: *Frontiers in Plant Science* 15 (June 2024), p. 1369696. DOI: [10.3389/fpls.2024.1369696](https://doi.org/10.3389/fpls.2024.1369696).
- [21] Cheng Zhou and Fanhuai Shi. “Perceptive Teacher: Semi-Supervised small object detection of Brassica Chinensis seedlings and pest infestations”. In: *Computers and Electronics in Agriculture* 229 (Feb. 2025), p. 109956. DOI: [10.1016/j.compag.2025.109956](https://doi.org/10.1016/j.compag.2025.109956).
- [22] Laura Gomez-Zamanillo et al. “Semi-Supervised Approach for Automatic Counting of Whiteflies With Small Annotated Dataset”. In: *IEEE Access* 13 (2025), pp. 55586–55598. DOI: [10.1109/ACCESS.2025.3552195](https://doi.org/10.1109/ACCESS.2025.3552195).
- [23] L. Minh Dang et al. “An efficient zero-labeling segmentation approach for pest monitoring on smartphone-based images”. In: *European Journal of Agronomy* 160 (Oct. 2024), p. 127331. DOI: [10.1016/j.eja.2024.127331](https://doi.org/10.1016/j.eja.2024.127331).
- [24] Waldemar Raaz et al. “Smart Checkpots–Proof of concept for a mobile pest and climate monitoring system in greenhouses”. In: *DGG-Proceedings* 11.8 (2023). DOI: [10.5288/DGG-PR-11-08-WR-2023](https://doi.org/10.5288/DGG-PR-11-08-WR-2023).

7 Glossary

- augmentation – Techniques that artificially increase dataset diversity by combining or altering images to improve model generalization.
- batches – Subsets of the dataset processed together during one iteration of training.
- class imbalance – When some classes appear much more frequently than others in the dataset, potentially biasing the model.
- COCO – A large-scale labeled image dataset commonly used for object detection and segmentation.
- color threshold based segmentation – A method to separate objects from the background based on color ranges.
- confidence – The probability assigned by a model to indicate how certain it is about a prediction.
- deep learning – Neural networks with multiple layers that learn features from data.
- domain shift – Differences between training and deployment data distributions that can reduce model performance.
- early stopping – A training strategy that stops training when the model performance on a validation set stops improving.
- epoch – Complete passes through the entire training dataset.
- F1-score – The harmonic mean of precision and recall, balancing false positives and false negatives.
- file checksums – Unique hash values used to verify file integrity; they are unambiguous identifiers for files.
- GPU – Graphics Processing Unit; used to accelerate large-scale computations for deep learning.
- grounding language-image model – Models that connect textual descriptions with images to detect or label objects.
- image processing – Processing images using algorithmic methods with manually designed features, as opposed to features learned automatically by deep learning models.

- IoU – Intersection over Union; a metric to measure the overlap between predicted and ground-truth bounding boxes.
- learning rate schedule – A strategy to adjust the optimizer’s learning rate during training to improve convergence.
- loss function – A function that quantifies the difference between predicted and true values, guiding the optimizer to adjust weights.
- loss weights – Weights assigned to different components of a loss function to prioritize certain tasks or classes.
- mean average precision – A standard object detection metric averaging precision across classes and confidence thresholds.
- mixup (included in augmentation) – A specific data augmentation technique that blends two images and their labels.
- NMS – Non-Maximum Suppression; a method to remove redundant overlapping bounding boxes in detection.
- mosaic – A data augmentation technique that combines multiple images into one, helping the model learn context and improve generalization.
- neural network – A computational model inspired by the brain, composed of interconnected nodes (neurons) that process data.
- optimizer – An algorithm that updates model parameters to minimize the loss function during training.
- padding – Adding extra pixels around images to make it divisible without remainder.
- patience – Number of epochs to wait without improvement before applying early stopping.
- precision – The proportion of positive predictions that are correct.
- precision-recall curve – A graph showing the trade-off between precision and recall at different confidence thresholds.
- pretrained – Models previously trained on large datasets, used as a starting point for new tasks.
- pseudo-labels – Labels generated by a model for unlabeled data; used in self-training methods to improve learning.
- recall – The proportion of true positives correctly detected by the model.

- self-training – A semi-supervised method where a model trains on unlabeled data using its own predictions as labels.
- semi-supervised learning – Training with a combination of labeled and unlabeled data to reduce labeling effort.
- stride – The step size used when sliding over an image in a window or tiling an image. If the stride is equal to the size of the window, no overlaps are created.
- stochastic gradient descent – An optimizer that updates model parameters using gradients computed on batches of data.
- supervised training / learning – Training a model with input-output pairs where the correct output (label) is known.
- teacher-student – A semi-supervised training setup where a “teacher” model generates labels for a “student” model to learn from.
- validation set – A dataset used to monitor model performance during training; different from the test set, which is only used for final evaluation.
- weights – The trainable parameters of a neural network that determine the influence of each input feature on the output.
- YOLO model – A real-time object detection neural network architecture designed for speed and accuracy.